

Programming Fundamentals – Lab 09 Tasks

Functions & Pointers in C — Spring 2026

Functions

Questions 1 – 6 (6 Questions)

Pointers

Questions 7 – 10 (4 Questions)

SECTION A — FUNCTIONS (Questions 1 to 6)

Q1

Function

Hospital Emergency Room — Patient Triage Score

SCENARIO

A hospital's emergency room receives dozens of patients every hour. To prioritize treatment, nurses assign each patient three scores: severity of injury (1–10), age risk factor (1–10), and vitals stability (1–10). A higher combined weighted score means the patient needs immediate attention. The hospital needs a software tool that computes this triage score automatically so staff can focus on care rather than manual calculations.

TASK

Write a C function `triageScore(int severity, int age, int vitals)` that accepts the three scores and returns a weighted priority score calculated as: $(\text{severity} * 0.5) + (\text{age} * 0.3) + (\text{vitals} * 0.2)$. In `main()`, take all three scores as user input, call the function, and print a message — if the score exceeds 7.0, print "Immediate attention required"; if between 4.0 and 7.0, print "Moderate priority"; otherwise print "Can wait". Use a float return type.

HINT

Use float `triageScore(int, int, int)` and declare it before `main()`. Multiplying integer parameters by float constants inside the function will promote them to float automatically.

Q2

Function

Online Retail Store — Discount and Invoice Generator

SCENARIO

An online clothing store applies different discount rates based on a customer's membership tier: Bronze gets 5%, Silver gets 10%, Gold gets 20%, and Platinum gets 30%. At checkout, the system must calculate the final payable amount after the discount and display an itemized invoice. A flat delivery charge of Rs. 150 is added if the order value after discount is below Rs. 2000.

TASK

Write two C functions: (1) `applyDiscount(float price, int tier)` — takes the original price and membership tier (1=Bronze, 2=Silver, 3=Gold, 4=Platinum) and returns the discounted price using if-else. (2) `printInvoice(float original, float discounted)` — a void function that prints the original price, discount amount, delivery charge if applicable, and the final total in a formatted invoice. Call both from `main()` with user-provided values.

HINT

Separate concerns: one function returns a value, the other just prints. Pass the return value of `applyDiscount()` directly to `printInvoice()`. Use `%.2f` for currency formatting.

Q3 Function	Cricket Analytics App — Batting Performance Evaluator
SCENARIO	<i>A cricket analytics company tracks the runs scored by a batsman in 10 consecutive matches. At the end of a series, the coaching staff wants to know: total runs scored, average runs per match, highest individual score, and the number of matches where the player scored above the average. This data helps decide whether a player should be retained for the next series.</i>
TASK	Write a C program with three functions, all accepting the scores array and its size: (1) totalRuns(int arr[], int n) — returns the sum. (2) highestScore(int arr[], int n) — returns the maximum value. (3) aboveAverage(int arr[], int n, float avg) — returns the count of elements greater than avg. In main(), input 10 match scores, call all three functions, compute the average from the total, and display a complete performance report.
HINT	Arrays decay to pointers when passed to functions — no size information is carried, so always pass n separately. Cast the total to float before dividing to get a decimal average: (float)total / n.

Q4 Function	Smart ATM — Secure PIN Validation System
SCENARIO	<i>A bank's ATM software must authenticate users before allowing any transaction. The system stores a 4-digit PIN for each account. When a user inserts their card, they are given up to 3 attempts to enter the correct PIN. If all 3 attempts fail, the card is blocked and a warning message is displayed. Each failed attempt must show how many tries remain.</i>
TASK	Write a C function validatePIN(int storedPIN, int enteredPIN) that returns 1 if the PINs match and 0 otherwise. In main(), set a hardcoded storedPIN = 4729. Use a loop that allows up to 3 attempts, calling validatePIN() each time. On success, print "Access granted. Welcome!" and exit the loop. After 3 failures, print "Card blocked. Contact your bank." Display remaining attempts after each wrong entry.
HINT	Use a for loop with a counter variable. Use break to exit early on success. The function only compares — all the loop logic and messaging stays in main().

Q5 Function	Power Plant Monitor — Unit Conversion Pipeline
SCENARIO	<i>A power plant monitoring system receives energy readings in Kilowatt-hours (kWh) from sensors. These readings need to be converted into three different units for three departments: Megajoules (MJ) for engineering, BTUs for international operations, and Calories (kcal) for the environmental report. The conversion formulas are: 1 kWh = 3.6 MJ, 1 kWh = 3412.14 BTU, 1 kWh = 859,845 calories.</i>
TASK	Write three separate C functions: toMegajoules(float kwh), toBTU(float kwh), and toCalories(float kwh), each returning the converted float value. In main(), take the kWh reading as input, call all three functions, and print a formatted conversion report. Then

	demonstrate function call chaining — pass the return value of toMegajoules() directly inside a printf() statement without storing it in a variable first.
HINT	Function return values can be used directly inside printf() as arguments: printf("MJ: %.2f", toMegajoules(kwh)); — this demonstrates that function calls are expressions in C.

Q6 Function	Restaurant Billing — Multi-Table Revenue Summary
SCENARIO	<i>A restaurant has 5 tables. At the end of each night, the manager wants to know the total revenue earned across all tables, which table generated the most revenue, and whether the night was profitable — defined as total revenue exceeding Rs. 10,000. Each table's bill amount is entered by the cashier at closing time.</i>
TASK	Write a C program with: (1) totalRevenue(float bills[], int n) — returns the sum of all table bills. (2) bestTable(float bills[], int n) — returns the 0-based index of the table with the highest bill. (3) isProfitable(float total) — returns 1 if total exceeds Rs. 10,000, 0 otherwise. In main(), input 5 bill amounts into an array, call all three functions, and display a closing summary showing total revenue, which table performed best (displayed as Table 1 to 5), and the profitability verdict.
HINT	In bestTable(), track the index of the maximum, not just the value — initialize int maxIdx = 0 and update it when a larger value is found. Add 1 when printing the table number since arrays are 0-indexed.

SECTION B — POINTERS (Questions 7 to 10)

Q7 Pointer	Exam Results System — In-Place Grade Scaling
SCENARIO	<i>A university professor discovers after a difficult midterm that the highest mark in the class was only 60 out of 100. To make grading fair, she decides to scale all marks using the formula: new mark = (original mark / highest mark) x 100. This scaling must be applied directly to the original array — no second array is allowed — and the program must use pointer arithmetic to access and modify every element.</i>
TASK	Write a C program that stores marks for 6 students in an integer array. Declare a pointer int *p = arr. Use a loop with pointer arithmetic *(p + i) to first find the highest mark, then in a second loop, scale each element in-place using the formula above (use float arithmetic, then cast back to int). Print the original array before scaling and the updated array after, using pointer notation throughout — no array indexing like arr[i] is allowed.
HINT	Scaling formula in code: *(p + i) = (int)(((float)* (p + i) / highest) * 100); — use two separate loops, one for finding max and one for applying the formula. Never use arr[i] anywhere.

Q8	IoT Sensor Hub — Multi-Type Data Logger
-----------	--

Pointer	
SCENARIO	<i>An IoT device collects three types of readings from different sensors in a smart factory: an integer from a vibration sensor (vibrations per second), a float from a temperature sensor (degrees Celsius), and a character code from a status sensor ('N' for Normal, 'W' for Warning, 'C' for Critical). The embedded system has a single generic pointer variable that must be reused across all three sensors to conserve memory resources.</i>
TASK	Write a C program declaring one void *sensor pointer. Assign it to an int variable (vibrations = 847), print the value by casting to (int *). Reassign to a float variable (temperature = 73.6), print using (float *) cast. Reassign to a char variable (status = 'W'), print using (char *) cast. After each print, also print the memory address stored in the pointer using %p. Finally, write a conditional that reads the status character through the void pointer and prints an appropriate alert message for each of the three status codes.
HINT	Dereference pattern: *(int *)sensor. For the conditional: if(*(char *)sensor == 'W'). Print addresses with %p and cast the pointer to (void *) in printf for correctness.

Q9 Pointer	School Attendance System — Weekly 2D Record Traversal
SCENARIO	<i>A school tracks daily attendance for 4 students over 5 school days. Attendance is recorded as 1 (present) or 0 (absent) in a 2D grid where rows represent students and columns represent days Monday through Friday. The attendance officer needs a program that uses a pointer to traverse the entire 2D grid and calculates each student's total attendance count for the week, flagging any student with fewer than 3 days present as being at risk.</i>
TASK	Write a C program with a 2D array int attendance[4][5] initialized with a mix of 0s and 1s. Declare a pointer int (*p)[5] = attendance. Use nested loops — the outer loop moves across rows using (p + i), the inner loop accesses columns using (*(p + i))[j] — to compute each student's weekly total. Print a report: for each student (Student 1 to 4), print their day-by-day attendance pattern and total days present, then append "At risk" if the total is below 3.
HINT	int (*p)[5] is a pointer to an array of 5 integers — the parentheses are critical. (p + i) points to row i. Dereference with *(p + i) to get the row, then index the column with [j] to get each element.

Q10 Pointer	Game Engine — Runtime Combat Action Dispatcher
SCENARIO	<i>A 2D combat game engine needs to handle different attack types that a player selects during their turn: a basic attack (deals damage directly), a power strike (multiplies damage by 2.5), a heal (restores 20 HP regardless of damage), and a poison attack (deals half damage and prints a status effect message). The battle engine must invoke the correct action at runtime based on the player's menu selection — without using a long if-else chain inside the combat loop itself.</i>

TASK	Write four C functions: <code>basicAttack(int dmg, int hp)</code> , <code>powerStrike(int dmg, int hp)</code> , <code>heal(int dmg, int hp)</code> which ignores <code>dmg</code> and adds 20 to <code>hp</code> , and <code>poisonAttack(int dmg, int hp)</code> which deals <code>dmg/2</code> and prints a poison message — all returning the new HP as <code>int</code> . Declare a function pointer <code>int (*action)(int, int)</code> . In <code>main()</code> , set <code>hp = 100</code> and <code>dmg = 25</code> . Show a menu, assign <code>action</code> to the appropriate function based on user input (no <code>if-else</code> inside the call), call <code>action(dmg, hp)</code> , update <code>hp</code> with the result, and repeat this for 3 consecutive turns.
HINT	Assign without calling: <code>action = basicAttack;</code> (no parentheses after the name). Call through the pointer: <code>hp = action(dmg, hp);</code> . All four functions must share the exact same signature <code>int(int, int)</code> for the pointer to be compatible.